Advanced Pygame Zero Cheat Sheet

CS4040 · Unit 5

Assets

Folder	Holds	File types
`images`	pictures for Actors	`.png`
`sounds`	sound effects	`.wav` or `.ogg`
`music`	background music	`.ogg` or `.mp3`

Asset names follow the same rules as Python variables: all lowercase, no spaces, only letters, numbers and underscores, and they must begin with a letter. Refer to an asset by its name without the extension: `"alien"`, not `"alien.png"`.

This module adds two more folders: keep your spritesheets in `sprites`, and your tilemaps, tilesets, and tilesheets in `maps`.

Actors

Actor extensions

Additional functionality added to Actors:

```
player.angle  # gets current angle in degrees
player.angle += 1  # increments angle by 1
player.scale  # gets current Actor scaling factor
player.scale = 2.0  # sets the Actor scaling factor
player.flip_v = True  # flips the Actor vertically
player.flip_v = False  # turns off vertical flip, default
player.flip_h = True  # flips the Actor horizontally
player.flip_h = False  # turns off horizontal flip, default
```

```
player.opacity = 0.6  # sets opacity, from 0.0 to 1.0
player.visible = False  # hides it; still moves and collides
```

Collision rectangle

A custom collision rectangle is centered on the Actor. The first line makes it 10 wide and 20 tall; the second makes it 50 wide and 20 tall, with its center moved by (5, −5).

```
player.collision_rect = (10, 20)  # (width, height)
# (width, height, offset_x, offset_y)
player.collision_rect = (50, 20, 5, -5)
```

In `draw()`:

```
player.draw_collision_rect()  # outlines it in red, for debugging
```

Sprites

What is a Sprite?

A `Sprite` is one animation: its frames, how fast to play them, and whether to loop. Give an Actor a Sprite instead of an image name, and it plays the animation by itself. Docs: Sprite Animations

Sprite from a list of images

List image names from the `images` folder, in order. Only `fps` and `loop` can be given.

```
# a Sprite made from a list of images in the images folder
alien_walk = Sprite(["alien_beige_walk1", "alien_beige_walk2"], fps=6)
```

```
alien = Actor(alien_walk)  # a Sprite instead of an image name
```

Sprite sheets

A **sprite sheet** is one image that holds every frame of a character's animations, laid out in a grid. Each **row** is one animation (idle, walk, jump, ...), and every frame is the same size. Rows and frames are counted from 0, starting at the top left.

Keep sprite sheets in the `sprites` folder. Each sheet there has a `.md` file listing its frame size and what is on each row.

Sprite Studio turns art from anywhere (sheets, GIFs, Aseprite files, folders of frames) into a sprite sheet that works here, and writes the `Sprite(...)` code for you.

Sprite from a sprite sheet

```
# Sprites made from rows of a sprite sheet in the sprites folder
idle = Sprite("rabbit", frame_width=16, frame_height=16, row=0, frames=2, fps=3)
walk_left = Sprite("rabbit", frame_width=16, frame_height=16, row=6, frames=2,
fps=8)
walk_right = Sprite("rabbit", frame_width=16, frame_height=16,row=7, frames=2,
fps=8)
```

- `image_name`: sprite sheet filename in the `sprites` folder
- `frame_width`, `frame_height`: size of each frame, in pixels
- `row` (optional): row to start from, counting from 0. Default `0`.
- `frames` (optional): number of frames. Default: the whole row.
- `fps` (optional): frames per second. Default `10`.
- `loop` (optional): `False` plays once. Default `True`.
- `transparent_color` (optional): color to make see-through.
- `spacing`, `margin` (optional): gaps between frames and around the sheet, in pixels.

Playing and switching Sprites: `actor.sprite`

Give the Actor a Sprite when you make it. Set `actor.sprite` to switch animations. Setting the Sprite that is already playing does nothing, so it is safe to do every frame.

```
rabbit = Actor(idle)  # starts by playing idle
rabbit.scale = 3  # pixel art is small, so make it bigger
```

```
if keyboard.left:
    rabbit.x -= 2
    rabbit.sprite = walk_left  # switch to another Sprite
elif keyboard.right:
    rabbit.x += 2
    rabbit.sprite = walk_right
else:
    rabbit.sprite = idle  # safe every frame: restarts only if it changed
```

Use `actor.sprite` for Sprites and `actor.image` for image names. Whichever you set last is what the Actor shows.

Advanced Sprites

Playing once: `loop=False`, `animation_finished`

A Sprite made with `loop=False` stops on its last frame. `animation_finished` is `True` once it has, so you can go back to another animation. `fox.sprite == hurt` checks which animation is playing.

```
idle = Sprite("fox", frame_width=24, frame_height=16,
              row=0, frames=5, fps=6)
hurt = Sprite("fox", frame_width=24, frame_height=16,
              row=4, frames=5, fps=8, loop=False)  # plays once
fox = Actor(idle)
```

```
if fox.sprite == hurt and fox.animation_finished:  # hurt has ended
    fox.sprite = idle  # go back to idle
```

Pausing, restarting, and frames

```
def on_key_down(key):
    if key == keys.SPACE:
        fox.sprite = hurt  # replays if it has already finished
    if key == keys.P:
        if fox.animation_paused:  # True while paused
            fox.resume_animation()  # carry on from the same frame
        else:
            fox.pause_animation()  # freeze on the current frame
    if key == keys.R:
        fox.reset_animation()  # play again from the first frame
```

```
screen.draw.text(f"frame {fox.frame}", (10, 10))  # frame number, from 0
```

Setting `fox.frame` jumps to that frame. While the whole game is paused with `game.pause()`, every Actor holds its frame.

Fullscreen

- Set `FULLSCREEN = True` at the top of your game to start in fullscreen.
- Press **F11** to toggle any game between fullscreen and a window.

Tile Maps

Tilesheets, tilesets, and tilemaps

- **Tilesheet**: a `.png` image file that contains many small images (tiles) arranged in a grid. Each tile is a piece of the world (grass, wall, water, etc.). This is the art asset created by the artist.
- **Tileset**: a `.tsx` file created in Tiled that defines how to use a tilesheet. It tells Tiled things like tile size, spacing, and any extra data. It references a tilesheet. (*Tileset* and *tilesheet* are often used interchangeably.)
- **Tilemap**: a `.tmx` file created in Tiled that defines the actual level layout. It uses one or more tilesets and organizes tiles into layers (ground, walls, hazards, etc.). This is the level design.

Note: the tilesheet, tileset, and tilemap must all be stored in the `maps` folder of your project.

Make maps in Tiled. Docs: Tile Maps

Tile map constants

Make your own constants and use them to compute the window size.

```
# our tile map constants
TILE_SIZE = 16  # length of a tile, in pixels
MAP_WIDTH = 30  # width of the tilemap, in tiles
MAP_HEIGHT = 20  # height of the tilemap, in tiles
SCALE = 2  # global scaling factor
# can use to compute screen dimensions
WIDTH = TILE_SIZE * MAP_WIDTH * SCALE
HEIGHT = TILE_SIZE * MAP_HEIGHT * SCALE
```

Loading a tilemap: `load_tile_map()`

Use `load_tile_map()` to import your map made in Tiled into your game based on tile map layers.

- `tmx_name`: path to the TMX file defining the tilemap, created in Tiled.
- `scale`: scaling factor applied to the map for display in Pygame.

Get each layer by its name in Tiled, like `level["walls"]`. A layer works like a list of tiles, and every tile is an Actor, so `collidelist()` and `for` loops work on it.

Using different layers allows you to treat each kind of tile differently.

```
level = load_tile_map("tl1.tmx", scale=SCALE)
ground = level["ground"]  # ground / background scenery to draw
walls = level["walls"]  # obstacles for player to collide with
hazards = level["hazards"]  # "bad" objects for player collision
collectables = level["collectables"]  # "good" objects to collect
```

The map, layers, and tiles

A tile map is made of three kinds of thing, each inside the one before:

- **The map** (`level`) holds every layer.
- **A layer** (`walls = level["walls"]`) is a list of tiles.
- **A tile** (`wall = walls[0]`) is one Actor on the map.

Each method belongs to one of them. The variable before the dot tells you which: `level.draw()` is the map, `walls.get_tile(...)` is a layer, `wall.set_tile_image(...)` is a tile.

Drawing a tilemap: `level.draw()`

Draws every layer in Tiled's layer order, from the bottom of Tiled's **Layers** list to the top: background first, foreground last. To change what's in front, drag the layers into a new order in Tiled. Draw the player after the map so it is on top.

```
level.draw()  # draws every visible layer, in Tiled's layer order
rabbit.draw()  # draw the player after the map, so it is on top
```

Layers hidden in Tiled aren't drawn, but still collide. Hide or show a layer in code with `hazards.visible = False` or `True`.

Tile Map Collisions

Wall collisions

Example of how to handle collisions with walls. Repeat the same steps for movement in each cardinal direction, putting the edge you moved against back on the wall.

```
# left keyboard button pressed
if keyboard.LEFT:
    rabbit.x -= 2  # move to left
    rabbit.sprite = walk_left  # change to left sprite
    # check for collision
    collision_index = rabbit.collidelist(walls)
    # did collision occur after moving to left?
    if collision_index != -1:
        # if so, put character's left edge on right edge of wall
        rabbit.left = walls[collision_index].right
```

Collecting items

`collidelist()` gives the index of the tile you touched in the layer. `pop()` that index to take it off the map.

```
# touching a collectable? take it off the map
item_index = rabbit.collidelist(collectables)
if item_index != -1:
    collectables.pop(item_index)
# collected them all?
if len(collectables) == 0:
    game.win()
```

Hazards

```
# touching any hazard ends the game
if rabbit.collidelist(hazards) != -1:
    game.lose()
```

Changing the Map with Code

Creating tiles: `level.create_tile()`

Columns and rows count from 0, starting at the top left. They mean two different grids:

- **The map** is a grid of squares. `col, row` says which square, like `(5, 2)`. Each layer holds at most one tile per square.
- **The tileset** is a grid of images. `tile_col, tile_row` says which image, like `(3, 8)`.

The **map** creates new tiles. `level.create_tile(tile_col, tile_row)` gives a new tile showing that image from the tileset. To find an image's `tile_col` and `tile_row`, count across and down the tilesheet from 0.

```
# a new tile showing the image at col 3, row 8 of the tileset
new_wall = level.create_tile(3, 8)
```

If your map uses only one tileset, `create_tile()` uses it automatically. If it uses more than one, name the tileset to use, as it is named in Tiled: `level.create_tile(3, 8, tileset="town")`.

Adding and removing tiles: `walls.set_tile()`, `walls.remove_tile()`

A **layer** adds and removes tiles by square. A new tile isn't on the map until you add it. `set_tile()` replaces any tile already in that square, and `remove_tile()` does nothing if the square is already empty.

```
walls.set_tile(5, 2, new_wall)  # put it at col 5, row 2 of the map
```

```
walls.remove_tile(5, 2)  # take away the tile at col 5, row 2, if any
```

To remove a tile you touched, `pop()` the index from `collidelist()`.

Is there a tile there? `walls.get_tile()` and `if wall:`

`get_tile(col, row)` gives back the tile in a square, or **`None`** if the square is empty. `None` means "nothing", and it has no methods, so using it crashes:

```
wall = walls.get_tile(0, 0)  # an empty square, so wall is None
wall.set_tile_image(4, 8)  # crash! None has no set_tile_image()
```

`if wall:` is `True` for a tile and `False` for `None`, so the code inside only runs when there is a tile:

```
wall = walls.get_tile(2, 5)  # a tile, or None if the square is empty
if wall:  # only if there is a tile there
    print(wall.map_col, wall.map_row)  # every tile knows its square
```

Changing a tile's image: `wall.set_tile_image()`

A **tile** can change which tileset image it shows. Find it with `get_tile()`, check it's there, then change it:

```
wall = walls.get_tile(2, 5)  # find the tile at col 2, row 5
if wall:
    wall.set_tile_image(4, 8)  # show image col 4, row 8 instead
```

Pixels and squares: `level.pos_to_map()`, `level.map_to_pos()`

The **map** converts between pixels and squares. `level.pos_to_map(x, y)` gives the `col, row` of the square at a position in pixels, such as a mouse click or the player. `level.map_to_pos(col, row)` gives the position of the center of a square.

```
col, row = level.pos_to_map(rabbit.x, rabbit.y)  # rabbit's square
```

```
rabbit.pos = level.map_to_pos(5, 9)  # middle of col 5, row 9
```

Clicking to build or knock down walls:

```
def on_mouse_down(pos):
    x, y = pos
    col, row = level.pos_to_map(x, y)  # the square that was clicked
    if walls.get_tile(col, row):  # is there a wall?
        walls.remove_tile(col, row)  # knock it down
    else:
        walls.set_tile(col, row, level.create_tile(3, 8))  # build one
```

Scrolling: `level.x`, `level.y`

For a map bigger than the screen, keep the player in the middle and move the map instead. Every tile moves with `level.x` and `level.y`, and collisions, `pos_to_map()` and `map_to_pos()` keep working.

```
def update():
    old_x, old_y = level.x, level.y
    if keyboard.left:
        level.x += 3  # move the map right, so the rabbit seems to walk left
    if keyboard.right:
        level.x -= 3
    if keyboard.up:
        level.y += 3
    if keyboard.down:
        level.y -= 3
    if rabbit.collidelist(walls) != -1:
        level.x, level.y = old_x, old_y  # hit a wall: put the map back
```

What Goes Where

What the map can do: `level`

The map holds every layer.

```
walls = level["walls"]  # get a layer by its name in Tiled
level.draw()  # draw every visible layer
new_wall = level.create_tile(3, 8)  # create a tile from the tileset
col, row = level.pos_to_map(x, y)  # pixels -> square
x, y = level.map_to_pos(col, row)  # square -> pixels (its center)
level.x -= 3  # scroll the whole map
```

What a layer can do: `walls`

A layer is a list of tiles.

```
first = walls[0]  # it's a list of tiles: [i], len(), for loops
index = rabbit.collidelist(walls)  # collisions work like any list
walls.pop(index)  # remove a tile by its index
wall = walls.get_tile(col, row)  # the tile in a square, or None
walls.set_tile(col, row, new_wall)  # put a tile in a square
walls.remove_tile(col, row)  # empty a square
walls.visible = False  # stop drawing it (it still collides)
```

What a tile can do: `wall`

A tile is one Actor on the map.

```
wall.pos  # it's an Actor: pos, x, y, colliderect(), ...
wall.map_col, wall.map_row  # its square on the map
wall.tile_col, wall.tile_row  # its image in the tileset
wall.set_tile_image(4, 8)  # show another image from the tileset
```